

Labwork 4: 2D Threads Configuration

Nguyen Duy Anh Tung

October 2, 2026

0.1 Introduction

In this labwork, we convert our previous 1D RGB-to-grayscale code into a 2D grid and block configuration. Instead of flattening the image into a 1D vector, we keep its native 2D matrix shape and observe how different block geometries affect running time. We also analyze hardware limits and answer the questions presented in the lecture slides.

0.2 Implementation

We measured the execution time of sequential CPU code against 1D and 2D CUDA kernels:

- **CPU:** Loops sequentially over rows (y) and columns (x), computing grayscale pixel by pixel using $g = \lfloor (R + G + B)/3 \rfloor$.
- **1D CUDA:** Flattens the image into size $(N, 3)$ and computes the linear index:

$$\text{tidx} = \text{threadIdx.x} + \text{blockIdx.x} \times \text{blockDim.x}$$

- **2D CUDA:** Keeps the original $(H, W, 3)$ shape and calculates pixel coordinates directly in 2D:

$$x = \text{threadIdx.x} + \text{blockIdx.x} \times \text{blockDim.x}$$

$$y = \text{threadIdx.y} + \text{blockIdx.y} \times \text{blockDim.y}$$

A simple check `if x < w and y < h` ensures threads beyond the image boundaries do not write into memory.

0.3 Experimental Results

The benchmark was run on an image with 199,875 pixels ($H \times W$). The sequential CPU version took **0.1967 s** (196.7 ms). GPU times are the average of 5 runs.

0.4 Discussion

- **1D vs 2D:** Both 1D and 2D grids achieve roughly the same execution time ($\approx 0.12 - 0.13$ ms). 1D has a slightly higher peak speedup because of simpler coordinate calculation, but 2D is much cleaner and more natural for working with images.
- **The (4, 4) Bottleneck:** The (4, 4) block performs noticeably worse (0.19 ms) because it only has 16 threads. As discussed in the slides, the GPU executes threads in warps of 32. Having fewer than 32 threads leaves half the warp inactive, wasting execution units.

Configuration	Threads/Block	Grid Size	GPU Time (ms)	Speedup
<i>1D Grid</i>				
32	32	6247	0.14	1449.8×
64	64	3124	0.12	1665.1×
128	128	1562	0.12	1636.7×
256	256	781	0.12	1662.5×
512	512	391	0.12	1680.8×
1024	1024	196	0.12	1587.6×
<i>2D Grid</i>				
(4, 4)	16	(94, 134)	0.19	1016.4×
(8, 8)	64	(47, 67)	0.13	1557.6×
(16, 16)	256	(24, 34)	0.13	1517.5×
(32, 16)	512	(12, 34)	0.12	1585.2×
(32, 32)	1024	(12, 17)	0.13	1503.1×

Table 1: Benchmark results for 1D and 2D CUDA configurations

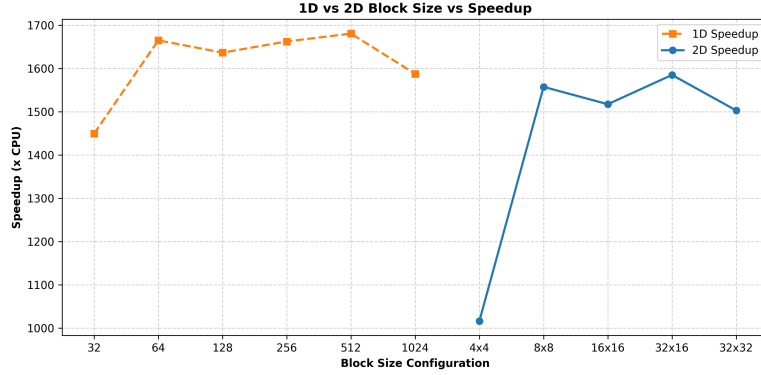


Figure 1: Speedup comparison between 1D and 2D block sizes

- **Hardware Limits and Crash:** Testing configurations above 1024 threads (such as $(64, 32) = 2048$ threads or > 1024 in 1D) causes an immediate `CUDA_ERROR_INVALID_VALUE` crash. This directly reflects the hardware restriction shown in the slides, where a single thread block is capped at 1024 threads.

0.5 Exercise Answers

- **Exercise 1 (Slide 24):** The best configuration is 16×16 . A 32×32 block has 1024 threads, exceeding the limit of 512 threads/block. An 8×8 block has 64 threads; with 8 blocks per SM, it only reaches $8 \times 64 = 512$ threads per SM (wasting half the 1024 limit). A 16×16 block has 256 threads, so 4 blocks fully populate the SM with $4 \times 256 = 1024$ threads.
- **Exercise 2 (Slide 25):** The configuration that yields the most threads is **512 threads/block**. Since the SM can hold at most 4 blocks, $3 \times 512 = 1536$ threads, which fully reaches the 1,536 thread limit. Other choices either produce fewer threads (e.g., $4 \times 256 = 1024$) or cannot fit more than one block ($1 \times 1024 = 1024$).
- **Exercise 3 (Slide 26):** The grid contains **2048 threads**. For a vector of size 2000 and 512 threads per block, we need $\lceil 2000/512 \rceil = 4$ blocks. This gives $4 \times 512 = 2048$ threads in total (where 48 threads remain idle due to boundary checks).